

Executive summary

Upstream GLIM has exactly one place that consumes GNSS — the `libgnss_global.so` extension module — and that module has no RTK gating of any kind. It accepts every received message uniformly and explicitly notes in its own header that it ignores GNSS observation covariance.

GLIM++ adds three independent RTK gates that close this gap on the Hitch Sensor Dome. Every place GNSS enters the system is now conditioned on RTK-fixed status (`STATUS_GBAS_FIX`) plus centimetre-grade position covariance ($\sigma \leq 0.10$ m by default).

Gate	Where	When	What it gates
Init pose gate	<code>glim_ros2/glim_ros.cpp::ins_pose_callback</code>	Once, at session start	Whether the optimizer is allowed to start at all
Per-message factor-bridge gate	<code>glim_ros2/glim_ros.cpp::try_publish_gnss_factor</code>	Every /pose message after init	Whether each GNSS sample becomes a soft factor in the global graph
Pre-recording pre-flight	<code>recording/sensor_recorder.py::RtkVerifier</code>	Before MCAP recording starts	Whether recording even begins (warn / hard / log)

Upstream GLIM has none of these. The bug pattern that motivates them is silent: there is no warning, log message, or error from upstream when GNSS quality degrades or has not yet reached RTK-fixed. The map just gets worse and the operator finds out by inspecting `T_world_utm.txt` or by overlaying the trajectory on satellite imagery and noticing it does not line up. The most common failure mode produced by the missing gates is the multi-lap z-drift documented in `GLIMplusplus/docs/multi_lap_loop_closure.md`: second-lap trajectories tilt upward into the sky because lap-1 GNSS-cold-start data was queued into the global alignment.

Every GNSS-touching code path — side-by-side

The table below enumerates every place GLIM consumes or produces data derived from GNSS — both upstream and in the GLIM++ fork. Each row gives the upstream behavior, the GLIM++ behavior, and a severity badge for what happens if upstream is used as-is on the Hitch Sensor Dome.

#	Code path	Upstream GLIM	GLIM++	Severity if upstream used as-is
1	Initial pose / gravity reference	Derives gravity from accelerometer mean over <code>initialization_window_size</code> (assumes vehicle stationary). No GNSS consumed at this stage.	INS-derived pose required. Three-stage gate: <code>status ≥ GBAS_FIX</code> , $\sigma \leq 0.10$ m, N consecutive stable samples.	Critical
2	libgnss_global.so factor input	Accepts any message on <code>gnss_topic</code> . No status check, no covariance check. Source comment: <i>'ignores the IMU-GNSS transformation and GNSS observation covariance.'</i>	Bridge republishes /pose only when latest /gps/fix shows RTK-class status + cm-grade σ . <code>gnss_global</code> never sees non-RTK-fixed input.	Critical
3	Factor weighting in gnss_global	Uniform <code>prior_inf_scale</code> applied to every accepted message regardless of source quality.	Same upstream behavior internally, but the bridge ensures only RTK-fixed messages enter, so the uniform weighting is correct in aggregate.	High
4	T_world_utm.txt export	Computed by least-squares alignment of submap origins to <code>gnss_global</code> 's queued positions. Inherits any garbage that passed (2).	Inherits a clean, RTK-fixed-only input stream from (2). The saved transform is reliably cm-grade.	Critical
5	RTK-loss handling mid-session	None. If RTK degrades, factors keep being added with the same <code>prior_inf_scale</code> using float-quality positions.	Bridge silently suspends during RTK-float / no-fix. Optimizer's LiDAR cost carries the trajectory through the gap. Resumes on re-fix.	High
6	GNSS factor density	One factor per <code>min_baseline</code> metres, regardless of arrival rate.	Same <code>min_baseline</code> semantics, but upstream of that the bridge throttles by RTK status. Net density is <code>min_baseline</code> -controlled when locked, zero when not.	Medium
7	Recording-time validation	None — upstream has no recorder.	RtkVerifier checks /gps/fix for <code>capture_window_s</code> before recording starts. Same gate as (1) and (2).	Operational
8	Multi-lap z-drift mitigation	Relies on whatever weight <code>prior_inf_scale</code> provides; cold-start data can dominate the optimizer. No way to decide what to trust.	Combination of (1), (2), and project-tuned <code>prior_inf_scale = [1e4, 1e4, 5e4]</code> (z 5x stronger than horizontal) reliably anchors lap 2+ to lap 1's altitude.	Critical
9	Configuration-level 'GNSS off'	Upstream default <code>prior_inf_scale: [0, 0, 0]</code> silently disables GNSS factors entirely. Many users never notice GNSS is off.	Default <code>prior_inf_scale: [1e4, 1e4, 5e4]</code> enables factors with the bridge's RTK gate as the per-message filter.	High

#	Code path	Upstream GLIM	GLIM++	Severity if upstream used as-is
10	Topic / message-type mismatch detection	None. Wrong message type → silent subscription that never fires.	Same upstream behavior, but the bridge republishes on a known type (PoseWithCovarianceStamped) so the mismatch can not recur.	High

Where upstream is silently buggy without RTK gating

Items (1), (2), (4), (5), (9), and (10) in the table are all **silent failures**. There is no warning, log message, or error from upstream when GNSS quality degrades or has not yet reached RTK-fixed. The operator finds out by inspecting `T_world_utm.txt` or by overlaying the trajectory on satellite imagery and noticing it does not line up.

Failure narrative — typical upstream session

First 30 to 120 seconds (cold-start): every `/gps/fix` queued by `gnss_global` has 3-10 m horizontal, 5-15 m vertical noise. The least-squares alignment that produces `T_world_utm` runs on a sliding window of these positions; the early samples permanently bias the alignment by metres.

Drive into a tunnel: RTK drops to no-fix, position dead-reckons via IMU. `gnss_global` keeps queuing positions (whatever arrives — including the dead-reckoned ones if the GNSS driver still publishes during outage). Each one adds a uniformly-weighted factor that drags the trajectory toward the dead-reckoning estimate.

Re-emerge from tunnel: RTK re-converges. The optimizer now has a mix of cm-grade factors and metre-grade factors with the same weight. Lap 2 alignment is loose at the metre scale.

Multi-lap session: the cumulative error compounds. The trajectory drifts up into the sky over time — the canonical *second-lap-tilts-to-the-sky* failure mode that motivated the GLIM++ fork's loop-closure tuning in the first place.

Critical places — ranked by impact

The most consequential differences between upstream GLIM and GLIM++, ordered by what breaks if you skip the gate.

Rank	Mechanism	Upstream GLIM	GLIM++	If you do not gate this
1	Init pose	Gravity-from-accel; no GNSS used	RTK-fixed INS pose required	Tilted world frame from frame zero — every map factor downstream is biased
2	First gnss_global factor	First arriving /gps/fix accepted	First RTK-fixed /pose accepted	T_world_utm aligned against pre-lock GNSS — saved transform is wrong by metres
3	Per-message factor stream	Every msg accepted equally	Only RTK-fixed accepted	Float-quality factors weighted equally to RTK-fixed → optimizer pulls trajectory off course
4	RTK-loss recovery	None — keeps queuing	Suspend, resume on re-fix	Tunnel / urban canyon corrupts the global anchor
5	Recording start	No validation	RtkVerifier pre-flight	Operator records before lock, does not find out until replay

Where GLIM++ still does NOT gate (intentional)

Worth flagging for completeness. The following are deliberate non-gates — each could be a useful enhancement, but none is necessary for correctness on the Hitch Sensor Dome.

1. gnss_global's internal weighting

The upstream `gnss_global_module.hpp` is not patched in GLIM++. The bridge filters inputs by status and covariance — that is a binary admit/reject decision, not a continuous downweight. Finer per-message weighting (RTK-fixed at 1 cm $\sigma \rightarrow$ weight 1.0; RTK-fixed at 3 cm $\sigma \rightarrow$ weight 0.3) would require modifying the upstream module. The bridge is sufficient and the upstream patch would be invasive.

2. IMU bias estimation

Does not consider GNSS quality. The optimizer's IMU factors trust IMU regardless of whether the most recent GNSS observation was centimetre or metre grade. This is correct (IMU bias is independent of GNSS quality) but worth noting.

3. Loop closure detection

Does not use GNSS distance as a sanity check. A `T_world_utm` mismatch between two submaps could in principle filter out false loops, but neither upstream nor GLIM++ uses it that way.

Source-code cross-reference

Exact files and functions for every claim in this report.

Feature	File	Symbol
Init RTK gate	GLIM_plusplus/glim_ros2/src/glim_ros/glim_ros.cpp	GlimROS::ins_pose_callback, ins_odom_callback, ins_init_timeout_tick
Factor bridge gate	(same file)	GlimROS::try_publish_gnss_factor
Pre-recording RTK gate	recording/sensor_recorder.py	class RtkVerifier, handle_rtk_outcome, RtkResult
Upstream GNSS consumer (only place GLIM touches GNSS)	GLIM_plusplus/glim_ext/modules/ mapping/gnss_global/ include/glim_ext/ gnss_global_module.hpp	GNSSGlobal::gnss_callback, push_gnss_data
Upstream init (now removed)	GLIM_plusplus/glim/src/glim/ odometry/ initial_state_estimation.cpp	NaiveInitialStateEstimation:: initial_pose (historically derived gravity from accel; now returns nullptr until external set_init_state is called)

Closing summary — one sentence

Upstream GLIM has exactly one place that consumes GNSS (`gnss_global`), it has zero RTK gating, and it explicitly notes in its own source that it ignores covariance — GLIM++ adds three independent RTK gates (init, per-message bridge, pre-recording) so that every place GNSS enters the system is conditioned on RTK-fixed status plus centimetre-grade σ .

Further reading. The full design of all three gates, including the C++ data flow, the bold-RED warning text, the ROS parameter list, and the sample log output for both success and failure cases, is in `GLIM_plusplus/docs/moving_start_initialization.md`. The downstream symptom that motivated the multi-lap z-drift fix (which depends on these gates working correctly) is in `GLIM_plusplus/docs/multi_lap_loop_closure.md`. The fork's complete change log relative to upstream `koide3/glim` is in `GLIM_plusplus/README.md`.

This report was prepared as part of the Hitch Sensor Dome project. GLIM is the work of Kenji Koide, Masashi Yokozuka, Shuji Oishi, and Atsuhiko Banno (AIST, Japan); upstream at github.com/koide3/glim. The fork's integration code in `GLIM_plusplus/` is part of the Hitch Sensor Dome project, designed and maintained by Dr. Allen Y. Yang (Hitch Interactive and University of California, Berkeley). Implementation testing by the Berkeley AI Racing Tech team.